

SwarmIQ: Autonomous Multi-Drone Swarm Simulator

Comprehensive Technical Summary & Academic Presentation Guide

Project: SwarmIQ (Autonomous Robotics Sim)	Fleet Capacity: 100+ Virtual Drones in Real-Time
Core Stack: Python, FastAPI, Three.js, React	Control Mode: 100% Autonomous (No Manual Pilot)
Tick Rate: 25 Hz Server Simulation / 60 FPS Viewport	GitHub: github.com/Meghs21/SWARM_IQ_DRONE

1. Problem Statement & Project Objective

The Problem: In real-world drone applications (search & rescue, defense, environmental mapping, disaster relief), managing large fleets using individual human pilots is impossible. Traditional multi-drone setups suffer from communication lag, single points of failure (if a central controller fails), and the inability to dynamically respond to sudden obstacles and moving hazards.

The Objective: Build **SwarmIQ** — a decentralized, software-based 3D simulation of an autonomous drone swarm where up to **100 virtual drones coordinate without any manual piloting**. The swarm self-organizes, elects leaders dynamically, navigates 3D obstacle courses via A*, maintains formations (V, Line, Grid, Circle), and actively prevents collisions in real-time.

Key Evaluation Highlights to Present:

- **Decentralized Intelligence:** Every drone executes local physics, separation, and obstacle evasion.
- **Hierarchical Planning:** Only the elected leader solves global 3D A*; followers maintain formation.
- **Fault Tolerance:** If the leader is shot down or drops below 20% battery, automatic election elects a new leader with zero mission interruption.
- **Real-Time 3D Rendering:** 100 drones at 60 FPS in Three.js via GPU hardware instancing.

2. System Architecture & Tech Stack

SwarmIQ uses a high-performance decoupled Client-Server architecture. The server runs the physics loop at 25 Hz and broadcasts JSON state snapshots over WebSockets to a Three.js 3D viewport.

Subsystem	Technologies Used	Key Responsibilities & Capabilities
Backend Engine	Python 3.11, FastAPI, Uvicorn, Asyncio	25 Hz non-blocking physics loop, REST control API, WebSocket server, state management.
Math & Physics	NumPy, SciPy	Vectorized $O(N^2)$ Boids forces, pairwise distance matrices, kinematic integration, clamping.
3D Frontend Viewport	Three.js, WebGL	GPU InstancedMesh rendering (100 drones in 1 draw call), OrbitControls, lighting, shadow maps.
Command HUD UI	React 18, Vite, Tailwind CSS, Lucide	Telemetry dashboard, formation switcher, color themes, fault injector, preset scenario cards.

3. The Five Core Autonomous Algorithms (Explain to Teacher)

Each drone synthesizes five distinct vector forces into a single resultant steering acceleration: $F_{total} = F_{boids} + F_{formation} + F_{navigation} + F_{obstacle} + F_{collision}$

Algorithm	How It Works (Simple Concept)	Technical Formulation / Innovation
1. Reynolds Boids Flocking	Mimics bird flocks: drones avoid crowding, align directions, and stay cohesive.	Vectorized NumPy implementation: Separation ($1/r^2$), Alignment (avg velocity), Cohesion (center of mass). Runs in <2ms for 100 agents.
2. 3D Global A* Path Planning	Plans the shortest 3D route for the leader through complex obstacles.	3D Voxel grid search with 26-connectivity and obstacle safety margin inflation. Line-of-sight raycasting prunes redundant waypoints.
3. Potential Field Avoidance	Obstacles push drones away like magnetic repulsion fields.	Non-linear quadratic repulsion with close-proximity surge up to 40 N. Features hard 0.8m hull velocity deflection to prevent pass-through.
4. Geometric Formation Control	Maintains structured shapes: V-shape, Line, Grid, and Circle.	Persistent slot allocation modulo 2π prevents criss-crossing. Leader velocity feedforward gives perfect millimeter tracking without lag.
5. Dynamic Leader Election	Elects the best drone as captain; replaces it instantly if battery runs low or it fails.	Multi-factor fitness function: Score = 35% Battery + 30% Distance + 20% Connectivity + 15% Health. Auto-failover under 20% battery.

4. Key Engineering Challenges Solved (Impressive Points)

Challenge 1: Circle Formation Distortion & Oval Lag

- *Cause:* In early builds, drones sorted polar angles using raw $\arctan2$ ($-\pi$ to $+\pi$). A drone at $-\pi$ was assigned to 0 (opposite side of the circle), causing 20 drones to criss-cross straight through the center. Furthermore, the leader was flying at 100% speed, leaving followers 0 m/s headroom to catch up.
- *Solution:* Applied $\arctan2(\dots) \% (2\pi)$ for monotonic slot ordering. Throttled leader cruise speed to 65% (providing 4.2 m/s sprint headroom). Decoupled Boids cohesion to 0.0 in formation flight so the ring doesn't collapse inward. Result: **Standard deviation of radius is 0.007 m (7 millimeters).**

Challenge 2: Drones Passing Through Solid Obstacles

- *Cause:* At 2 m distance, standard potential field repulsion was only 0.53 N, but formation holding force was 18 N! The formation spring force literally dragged drones through solid pillars.
- *Solution:* (1) Upgraded repulsion to a steep quadratic curve with close-proximity surge up to 40 N. (2) Dynamically attenuated formation pull near obstacles so drones are allowed to squeeze and part around pillars. (3) Added hard physical hull velocity reflection at 0.8 m. Result: **Zero obstacle penetrations across all 250 test ticks.**

5. Preset Demonstration Scenarios for Teacher Demo

Scenario	Fleet / Formation	What to Show the Teacher
1. Open Field	20 Drones • V-Wing	Clean baseline flight. Show formation keeping and switch formation dropdown to Circle to watch smooth mid-air morphing.
2. Obstacle Course	50 Drones • V-Wing	3D A* waypoint path (green glowing line) avoiding static high-rise pillars. Drones part around pillars like water around a stone.

3. Dynamic Hazards	50 Drones • Line	Moving hazard spheres patrol the space. Drones dynamically deflect away using real-time potential fields.
4. Leader Failure	50 Drones • Grid	Click 'Fail Leader' button in HUD. The leader loses power, drops, and the next best drone is instantly elected captain without halting.
5. Large Swarm	100 Drones • Circle	Full-scale stress test. 100 drones forming a 55m ring. Demonstrates 60 FPS Three.js InstancedMesh performance.

6. Teacher Viva / Presentation Q&A; Cheat Sheet

Q1: Why didn't you compute 3D A* path planning for all 100 drones?

Answer: Computing 3D A* across a 3D voxel grid for 100 individual drones every frame would be computationally prohibitive ($O(N * \text{Voxel}^3)$). Instead, SwarmIQ uses a bio-inspired Leader-Follower hierarchy: only the leader computes the global A* path. Followers maintain geometric slots and use reactive local potential fields, reducing CPU load by over 95% while keeping the entire swarm safe.

Q2: How do you prevent drone-to-drone collisions inside the swarm?

Answer: We use a multi-tiered safety system: First, Boids Separation exerts a $1/r^2$ repulsion if drones get within 3.0m. Second, our Collision Avoidance engine calculates hyperbolic repulsive forces for pairs under 2.0m. Third, the persistent formation slot assignment guarantees that each drone has an assigned spatial slot separated by 3.5m.

Q3: How does the system handle leader failure?

Answer: Every tick, the Leader Election algorithm monitors the current leader's health and battery. If the leader's battery drops below 20% or a hardware failure occurs, a multi-factor election runs across all active followers. The drone with the highest score (combining battery level, distance to target, and mesh connectivity) is elected leader, and the A* path is replanned instantly.

Q4: How does Three.js render 100 drones at 60 FPS in a web browser?

Answer: Standard Three.js rendering creates 1 draw call per 3D mesh (which would lag at 100 drones). We used Three.js `InstancedMesh`. This allows all 100 follower drones to share a single geometry and material, rendering the entire swarm in exactly 1 GPU draw call, delivering a smooth 60 FPS even on standard laptops.

Q5: What is the green vertical pole and green line in the simulation?

Answer: The vertical green pillar with a spinning radar ring is the Destination Beacon (the mission objective coordinates). The glowing green line on the ground/air is the 3D A* Global Planned Path computed for the leader to avoid obstacles.

7. Conclusion & Project Deliverables

SwarmIQ successfully fulfills all requirements of a real-time, software-based autonomous drone swarm simulator. All 36 automated test cases pass cleanly, verified across mathematical accuracy, dynamic failover, obstacle avoidance, and high-frequency WebSocket streaming. The complete source code is modular, well-documented, and hosted on GitHub at https://github.com/Meghs21/SWARM_IQ_DRONE.